

Chess Bot Project: Phase 4

Evaluation

Your bot from Phase 3 plays each turn by scoring its candidate moves with a small fixed formula: the value of the piece it captures, minus the value of the moving piece if the destination square is attacked by something cheaper. The formula works because chess punishes giving up material more than almost anything else, but it has a hard ceiling. Two captures of equal nominal value look identical to the bot, two quiet developing moves score zero alongside an aimless rook shuffle that also captures nothing, and a position with a passed pawn one square from queening looks no better than a passive position with the same material. Phase 4 lifts that ceiling by giving the bot a way to express which positions it actually wants to reach.

The idea is to separate two questions that the Phase 3 bot tangles together: what is the right move to play, and how good is the position you would land in? These are not the same question, and once we pull them apart the answer to the first becomes mechanical given an answer to the second. A function called `evaluate(board)` that takes a board state and returns a centipawn score from White's point of view is the central object of this phase; everything else in the bot becomes a thin shell around it. Phase 5's job will be to extend that shell into a multi-ply search, but the evaluation function itself, with its piece values and piece-square tables and mobility terms, is yours to shape in this phase, and the shape you give it is the personality your bot takes into the arena.

Up until now there has been one right answer for every test. Legal moves matched the FIDE rulebook, perft counts matched published reference numbers, the UCI loop accepted exactly the four commands it should. Phase 4 is the first phase where two correct implementations can disagree about what to play, and the leaderboard becomes the arbiter. Two students with thoughtful evaluation functions might both be objectively reasonable, and the head-to-head record will reveal whose values were better calibrated for the kind of positions that actually arise.

Setup

We are bumping the kit to v0.4.0, which introduces a new `evaluation.py` scaffold file, a library of canonical piece values and piece-square tables, and five new house bots with distinct personalities (`materialist`, `knightmare`, `edge_lord`, `crusader`, `hoarder`). Your existing `board.py` and `bot.py` are preserved.

1. Activate your venv and upgrade the kit.

```
cd chess-bot
source .venv/bin/activate
uv pip install --upgrade git+https://github.com/Eastbridge-Academy/chess-client-kit@v0.4.0
```

2. Pick up the new scaffold file.

```
chess-cli init
```

You should see `wrote evaluation.py` (the new file) alongside `skipping board.py` (already exists) and `skipping bot.py` (already exists). Open `evaluation.py` in your editor; the `evaluate` function is currently a stub.

Stage 13: Material and Mate

The simplest evaluator • Terminal positions

The simplest evaluation function you could possibly write counts the pieces on the board and weights them by how much they are worth. This is what every chess engine in history did first, including the very first ones in the 1950s, and even now in 2026 with neural network engines dominating the top of the rating list, the inner workings of those networks include a layer that does almost exactly this before all the more sophisticated stuff kicks in.

The piece values used to be the subject of mild academic argument. Lucena in his 1497 treatise wrote down something close to 1, 3, 3, 5, 9 for pawn through queen, scaling so that the pawn is the unit, and that approximation has held up for half a millennium because it captures something real about how the pieces actually interact in the average position. Modern engines work in centipawns, hundredths of a pawn, so the same values become 100, 300, 300, 500, 900. We use centipawns throughout this project because evaluation differences in real positions are usually smaller than one pawn, and we want the resolution to distinguish them.

The numbers you see in engines today are tuned more carefully than the classical 1-3-3-5-9. The most influential study is Larry Kaufman's 1999 paper *The Evaluation of Material Imbalances*, in which Kaufman, an American grandmaster who later consulted on Komodo and Rybka, derived piece values empirically from a large database of grandmaster games. The values fall out of the fit as 1, $3\frac{1}{4}$, $3\frac{1}{4}$, 5, $9\frac{3}{4}$ for pawn, knight, bishop (unpaired), rook, and queen, with a separate bishop-pair bonus of half a pawn for a side that has both bishops on the board. Several relationships drop out of the fit that the classical values only roughly suggest. Two rooks are slightly stronger than a lone queen, but the gap is much smaller than the classical 10 vs 9 would have you believe; the real margin is closer to a quarter of a pawn, since the queen sits at $9\frac{3}{4}$. An unpaired bishop and knight are essentially equal in value, and the bishop's reputation comes entirely from the half-pawn pair bonus. Two minor pieces come in just under rook-plus-two-pawns unless the two minors include the bishop pair, in which case they're a touch better. Kaufman published a simpler rounded set in 2012 (3.5 / 3.5 / 5.25 / 10) for over-the-board human use, but the 1999 fractional values are what engine programmers cite. Both tables are reasonable starting points, and your bot is welcome to use either, to mix them, or to settle on its own values once it has played enough games to have a feel for what works.

Piece	Classic	Kaufman
Pawn	100	100
Knight	300	325
Bishop	300	325
Rook	500	500
Queen	900	975
King	20000	20000

Both tables are exported from `chessdk` as `PIECE_VALUE_CLASSIC` and `PIECE_VALUE_KAUFMAN`; either is a single import away, or you can write your own dict literal and put the numbers wherever feels right. The king's value at 20,000 is a sentinel rather than a real assessment;

both kings are always on the board, so the contributions cancel and it never actually shows up in the material sum.

Material counting alone is enough to play recognizable chess, but it has one structural blind spot the evaluator has to handle separately. When the side to move has no legal moves and is in check, the game is over and the side to move has lost; this is checkmate, and a static count of pieces on the board has no way to see it (the mated player still has all of their pieces, including the king, sitting on their squares). When the side to move has no legal moves but is *not* in check, the game is also over but it is a draw; this is stalemate, and material counting again misses it. The evaluator has to recognize both, before getting to the material sum, because otherwise it will badly misread the position.

The convention is to return a very large value with the appropriate sign for checkmate, and zero for stalemate. The kit ships `MATE_SCORE = 1,000,000` as the canonical magnitude. Returning `+MATE_SCORE` unambiguously says “this position is a win for White,” returning `-MATE_SCORE` says “this position is a win for Black,” and the number is large enough to dominate any combination of normal evaluation terms while staying well within the range of comfortable integer arithmetic.

Step 1.

Open `evaluation.py`. Implement `evaluate(board)` to return a centipawn score from White’s point of view, with three branches:

- (i) If the side to move has no legal moves and is in check, the side to move has been checkmated; return `-MATE_SCORE` if the side to move is White (mated White means Black has won, so a strongly negative score from White’s POV), `+MATE_SCORE` otherwise.
- (ii) If the side to move has no legal moves and is not in check, the position is a stalemate; return 0.
- (iii) Otherwise, walk the squares of the board, sum each piece’s value with a `+1` for White pieces and a `-1` for Black, and return the total. The king’s contribution cancels out by symmetry, so you can either include it in the sum and rely on cancellation, or skip kings explicitly.

You may import `PIECE_VALUE_CLASSIC` or `PIECE_VALUE_KAUFMAN` from `chessdk`, or define your own. `MATE_SCORE` is exported by the same module.

Step 2.

Run the Stage 13 tests:

```
chess-cli test -k stage13
```

The material-direction tests check rough sign and order of magnitude on a battery of imbalanced positions: starting position close to zero, a queen up evaluating in the high hundreds, two rooks up evaluating in the high thousand, and so on. The terminal tests cover checkmate-against-Black (mate-magnitude positive), checkmate-against-White (mate-magnitude negative), and stalemate (near zero).

Stage 14: From Eval to a Bot

The eval-and-pick framework

With `evaluate` in hand, the bot's logic becomes a small loop that the rest of the project will keep returning to. The structure is: for each legal move, apply it, evaluate the resulting position, undo the move, and after every move has been visited return the one whose resulting position evaluated best for the side to move.

"Best" is asymmetric. White wants to maximize the score, since positive favors White, and Black wants to minimize it, since negative favors Black. The same evaluator serves both sides, because "score is from White's point of view" is a global convention rather than something each side computes separately. When the side to move is White, you pick the move whose resulting position is the highest-scoring; when it is Black, the lowest-scoring; in either case the choice is local to one ply of lookahead.

This is structurally identical to depth-1 minimax, which is what Phase 5 will extend by recursing further down the move tree. The pattern is so fundamental that the rest of the project keeps the outer loop intact and rewrites only the inner evaluation, or the depth at which the loop recurses, or the order in which moves are tried. The loop you write today is the spine of every bot in the project from here forward.

Three implementation notes. First, make and undo the move around every `evaluate` call so the board's state is unchanged when the loop exits. The `make_move/undo_move` pair from Phase 2 is doing real work here for the second time (the first was the hang detector in Stage 12), and any subtle bug in undo will surface as positions evaluating differently across iterations of the loop. Second, break ties at random. There will be many positions where two or more legal moves evaluate to exactly the same score, especially early in the game, and a deterministic tiebreaker would make your bot's first dozen moves identical from game to game, which is unhelpful both for testing and for the leaderboard. Third, the side to move on the *current* board determines whether you maximize or minimize; the side to move on the *post-move* board, which `evaluate` sees, has already flipped, but the convention that "positive favors White" is unchanged.

Step 1.

Restructure your Phase 3 `choose_move` in `bot.py` around the new evaluator. The Phase 3 gain term (the value of any captured piece) is subsumed by `evaluate(board_after_move)`, since the post-move material balance the evaluator sees already reflects the capture; but the Phase 3 hang penalty (the moving piece's value when its destination is attacked by something cheaper) has no equivalent inside the static evaluator, and at depth one the bot still cannot see the opponent's recapture without it. Keep the hang term as a depth-one stopgap. Phase 5's deeper search will subsume it structurally.

The full scoring rule for each move becomes:

- (i) Make the move on the board.
- (ii) Call `evaluate(board)` on the resulting position; call this the position score (from White's point of view).

- (iii) Ask `min_attacker_value` for the cheapest enemy attacker on the moving piece's destination square (this is the same helper you used in Stage 12); if it is smaller than the moving piece's value, set the hang penalty to the moving piece's value, otherwise to zero.
- (iv) Undo the move.
- (v) Combine the two into a single score: from the moving side's point of view the hang is a loss, so subtract it. In White-relative terms, that means `score = position_score - sign * hang`, where `sign` is +1 if White is to move and -1 if Black is to move.

Pick the move whose score is highest if White is to move on the original board and lowest if Black is to move. Break ties at random.

The imports you need are `evaluate` from your new `evaluation` module, plus `Move`, `WHITE`, your piece-values dict, `min_attacker_value`, and `opposite` from `chessdk`, plus `random`.

Step 2.

Run the Stage 14 tests:

```
chess-cli test -k stage14
```

The tests give your bot three mate-in-1 positions (back-rank mate, smothered mate, and a Black-to-move back-rank mate) and verify the bot picks the mating move. Because `evaluate` returns `MATE.SCORE` on the position after a mating move and a normal-magnitude score otherwise, the eval-and-pick loop should always prefer the mating move; if a test fails, the bug is almost certainly in the selection logic (forgetting to maximize vs minimize correctly when it is Black to move), not in the evaluator.

Step 3.

Play your first match against the new house bot at this stage:

```
chess-cli play --vs materialist -n 10
```

`materialist` is the Phase 4 calibration baseline: depth-1, material-only, no PSTs, no mobility, no king safety. With Stage 13's evaluator in place plus the hang penalty carrying over from Phase 3, your bot and Materialist are roughly even on the matchup: you both count material correctly, Materialist matches your hang behavior because its eval-and-pick at depth one happens to land on the same conservative moves a lot of the time, and most games end in long draws or tactical exchanges that go either way. Wins and losses should be roughly balanced. The next stage gives you the first real positional handle over Materialist.

The other house bots ship at v0.4.0 too, but they get more interesting after you have piece-square tables and mobility; we will work through them as we go.

Stage 15: Piece-Square Tables

PSTs • Where the values come from • From hand-tuning to neural nets

A material-only evaluator has a structural blindness: it cannot see that the same piece on different squares is worth different amounts. Every club-level player knows that a knight on e5 is dramatically better than the same knight on a8, that a passed pawn on the seventh rank is almost a queen, and that a bishop in the corner is mostly contributing to the décor. None of this lives in the material count. Piece-square tables, often shortened to PSTs, are the technique for putting it back in.

The data structure could not be simpler. For each piece type, we maintain a 64-entry table that maps a square to a centipawn bonus. When evaluating a position, you walk the pieces; for each piece you look up its square in the appropriate table and add the bonus to the running total, with the sign matching the piece's color. For Black pieces you also mirror the lookup square vertically, because a Black pawn on rank 2 is the analogue of a White pawn on rank 7 (both are one step from promoting on their own side's board). The mirror is exactly `sq(file_of(s), 7 - rank_of(s))`, and the kit exports a helper `pst_square(piece_kind, color, square)` that does it for you.

The numbers themselves come from a hand-tuned starting set published by Tomasz Michniewski in the early 2000s as the “Simplified Evaluation Function” on the Chess Programming Wiki. The set has been the canonical first PST for amateur engines for two decades, and it is the one the kit ships as the default. The values are modest by design (the largest non-zero entry is the pawn rank-7 bonus of +50), so a pawn one square from promoting is worth roughly 150 centipawns in total: 100 material plus 50 PST. Material remains the dominant term in the evaluator; the PST nudges the bot toward better squares without overriding the obvious material concerns.

The four most important tables, visualized as heat maps (green for positive, red for negative, with the centipawn value written in each cell):

Pawn

8	0	0	0	0	0	0	0	0
7	50	50	50	50	50	50	50	50
6	10	10	20	30	30	20	10	10
5	5	5	10	25	25	10	5	5
4	0	0	0	20	20	0	0	0
3	5	-5	-10	0	0	-10	-5	5
2	5	10	10	-20	-20	10	10	5
1	0	0	0	0	0	0	0	0
	a	b	c	d	e	f	g	h

Knight

8	-50	-40	-30	-30	-30	-30	-40	-50
7	-40	-20	0	0	0	0	-20	-40
6	-30	0	10	15	15	10	0	-30
5	-30	5	15	20	20	15	5	-30
4	-30	0	15	20	20	15	0	-30
3	-30	5	10	15	15	10	5	-30
2	-40	-20	0	5	5	0	-20	-40
1	-50	-40	-30	-30	-30	-30	-40	-50
	a	b	c	d	e	f	g	h

Bishop

8	-20	-10	-10	-10	-10	-10	-10	-20
7	-10	0	0	0	0	0	0	-10
6	-10	0	5	10	10	5	0	-10
5	-10	5	5	10	10	5	5	-10
4	-10	0	10	10	10	10	0	-10
3	-10	10	10	10	10	10	10	-10
2	-10	5	0	0	0	0	5	-10
1	-20	-10	-10	-10	-10	-10	-10	-20
	a	b	c	d	e	f	g	h

King (middlegame)

8	-30	-40	-40	-50	-50	-40	-40	-30
7	-30	-40	-40	-50	-50	-40	-40	-30
6	-30	-40	-40	-50	-50	-40	-40	-30
5	-30	-40	-40	-50	-50	-40	-40	-30
4	-20	-30	-30	-40	-40	-30	-30	-20
3	-10	-20	-20	-20	-20	-20	-20	-10
2	20	20	0	0	0	0	20	20
1	20	30	10	0	0	10	30	20
	a	b	c	d	e	f	g	h

Reading the heat maps: the pawn table strongly rewards advancement (rank 7 is worth +50 across the whole rank), and it penalizes the d2/e2 pair where pawns block the bishop and queen from coming out. The knight table is the most pronounced centralization table in the set; a knight in the very center has twenty centipawns of bonus, a knight in the corner has a fifty-point penalty, and the gradient between them is steep enough that knights on the rim really do feel grim. The bishop table prefers the long diagonals and the fianchetto squares. The king table is the middlegame version, which penalizes the king severely for leaving its castling corner, and rewards the corners themselves; the same logic flips in the endgame, where the king becomes an attacking piece and a different (separate) PST is preferred. The rook and queen tables, which we are not showing here but which are exported as `ROOK.PST` and `QUEEN.PST`, are gentler in magnitude; rooks have a small bonus on the 7th rank (where they cut into the enemy position) and queens have a mild preference for central squares.

The values you see above are not magic. They were chosen by Michniewski to encode the positional principles every chess teacher repeats, and they happen to play decent chess when used as the sole positional term in an evaluator. But they are one set of choices among many; the chess programming literature contains a great variety of alternative tables, and there is no reason to treat the canonical numbers as anything but a starting point. As your bot plays games, you will probably find positions where one of these entries seems wrong for the kind of game your bot wants to play, and tweaking the offending entry is a real and recommended customization.

Where do PSTs come from, historically? A short tour through the engineering history of evaluation is worth the digression, because it traces the entire arc of how computer chess evaluators have evolved over seventy years.

The 1950s through the 1980s were the era of hand-crafted evaluators. Programmers wrote down piece values and PST entries that felt right based on watching their programs play, and edited the values by hand whenever the program lost a game in an interesting way. Robert Hyatt, the author of the long-running open-source engine Crafty, has described his early process as essentially trial-by-error: type in a number, play a few hundred games, type in a different number, repeat. The values weren't optimized; they were apprenticed.

The 1990s saw the first systematic tuning. Programmers like Vincent Diepeveen and Bob Hyatt started using databases of completed games as ground truth: take a million positions where the eventual outcome is known, tweak the evaluator's parameters slightly, see whether the

new parameters predict the outcomes more accurately. The procedure is gradient descent in everything but name, and it produced PSTs noticeably better than the hand-tuned predecessors.

2008 brought a formalization. Peter Österlund, author of the open-source Texel engine, published a recipe on talkchess.com that became known as “Texel’s tuning method.” The recipe was precise: take a large database of quiet positions (positions with no captures pending, so the static evaluator is meaningful), label each with the actual game outcome the position came from, define the prediction error as a logistic-sigmoid distance between the evaluation and the outcome, and minimize the error by gradient descent on the evaluator’s parameters. The method was so robust that nearly every classical engine adopted it within a year or two, and Stockfish’s evaluator was Texel-tuned for the entire decade that followed.

2018: AlphaZero. Google DeepMind published the AlphaZero paper, in which the evaluation function was no longer a hand-written formula tuned over a database, but a deep convolutional neural network trained by self-play alone. There were no piece values, no PSTs, no mobility terms, just an opaque network that took a board as input and returned a single number. Within hours of self-play training, AlphaZero was playing chess at a level above Stockfish’s classical evaluator. The result was so striking that it took the community a few years to fully absorb what it meant for the discipline.

2020: NNUE. Yu Nasu, then a Japanese student working on a shogi engine, designed a small neural network architecture that was fast enough to use inside a classical alpha-beta search. The network, called the Efficiently Updatable Neural Network (NNUE), takes the position encoded as a vector of (piece type, piece square, friendly king square) features, runs it through a few fully-connected layers, and outputs a centipawn evaluation. The trick that gave NNUE its name was that as moves are made and unmade during search, the network’s first-layer activations can be updated incrementally rather than recomputed from scratch, which keeps the per-evaluation cost low enough to run millions of evaluations per second. Stockfish adopted NNUE in 2020 and immediately gained around 80 Elo on its previous version. Top engines today use NNUE-style neural evaluators almost universally.

A useful way to see NNUE is that it is the piece-square table generalized to its limit. A classical PST asks “where is this piece sitting?” and answers with a fixed number. NNUE asks “where is this piece sitting, where is every other piece, where is each king, what’s the phase of the game, and what’s the structure of the pawn chain?” and answers with a number that has been tuned on tens of billions of positions. The classical PST is the first-order approximation; NNUE is the higher-order correction. For Phase 4 we are emphatically in the first-order regime, but the family resemblance is the right way to read what we are doing.

Step 1.

Extend your `evaluate` to add a PST term to the material sum. For each piece on the board, look up its position in the appropriate PST and add the entry to the running total, with the sign matching the piece’s color (positive for White, negative for Black, so the overall convention of “positive favors White” stays consistent). For Black pieces, the table lookup uses the mirrored square `sq(file_of(s), 7 - rank_of(s))`.

The kit exports the PSTs in three forms, any of which is fine to use. Per-piece-kind tables (PAWN_PST, KNIGHT_PST, and so on) are 64-entry tuples indexed by `sq(file, rank)`. The dict

DEFAULT_PSTS is keyed by Kind, so a single `DEFAULT_PSTS[piece.kind][square]` expression covers the whole lookup. The helper `pst_square(piece.kind, color, square)` does the Black mirror for you and returns the entry directly; the choice between using it and writing your own mirror is a question of taste.

If you would rather supply your own PSTs (you saw a tuning experiment somewhere on the web, or you want to encode a particular playstyle), nothing stops you from defining them inline as tuples in `evaluation.py`. The default is just a useful starting point.

Step 2.

Run the Stage 15 tests:

```
chess-cli test -k stage15
```

The tests compare position pairs with identical material but different piece placement: a knight on e4 vs the corner of the board, a pawn on rank 7 vs rank 2, a bishop on the long diagonal vs the corner. In each case the better-placed piece should produce a higher evaluation from White's POV.

If the knight pair fails, the most common cause is a missing mirror for Black pieces (your code looks up the table directly for both colors, so a White piece on a8 and a Black piece on a8 receive the same PST contribution rather than mirrored ones); the symptom is that mirror-symmetric positions evaluate to zero even when one side's piece placement is clearly stronger. If the pawn-rank-7 test fails, double-check that you are adding the PST entry rather than the table's raw value; the entries are positive for advanced pawns, so White's contribution increases and Black's decreases as pawns advance.

Step 3.

Rematch the house bots, including the two that get more interesting now that you have a PST:

```
chess-cli play --vs materialist -n 10
chess-cli play --vs knightmare -n 10
chess-cli play --vs edge_lord -n 10
```

Materialist should now lose more often than it wins, though many games will still draw at the move cap because depth-one play cannot drive a positional edge home. You both count material correctly, but you alone know that pieces have preferred squares; when a game does develop into a recognizable middlegame, your slow accumulation of positional advantage compounds.

Knightmare is the same as Materialist except that it values knights at 500 centipawns (above rooks) and bishops at 200 (below 2 pawns). Its personality is immediately visible in its play: it will offer bishop-for-knight trades at every opportunity, sometimes for no reason. Your bot with sensible piece values should diagnose the mismatch within a few moves and start picking up its loose bishops.

Edge Lord uses the kit's default PSTs but with every value negated, so it rewards pieces on the edge of the board and penalizes the center. Knights on the rim, bishops in the corner,

king out for a stroll in the middlegame. It is a vivid demonstration that a PST is real and the direction of its values matters; against your correctly-signed PST, Edge Lord's pieces wander into bad squares and get picked off. The matchup is also a useful sanity check on your PST mirroring, since asymmetries between your White and Black play surface against an opponent making complementary mistakes.

Going Deeper. If you want to push your evaluator beyond the canonical PSTs, the standard route is to tune the values against your own game data using Texel's method. The recipe in short form: play a few thousand games of your bot against itself or against the house bots, recording every position and the eventual outcome. Pick a held-out set of, say, a thousand quiet positions, each labeled with the result (1 for a White win, 0 for a Black win, 0.5 for a draw). Define your loss as the sum over positions of $(\text{outcome} - \sigma(s))^2$, where s is your evaluator's score in centipawns and $\sigma(s) = 1/(1 + e^{-s/k})$ is a logistic sigmoid scaling the centipawn score into a probability of White winning (with k around 400). Now run gradient descent on the parameters of your evaluator (piece values, PST entries, mobility weight) to minimize the loss. The method is robust and forgiving, and a few hours of tuning on game data your bot generates can produce a measurably stronger evaluator. Österlund's original post on talkchess.com is the canonical reference; several follow-ups by Stockfish contributors and the chessprogramming wiki article on "Automated Tuning" go into more detail on the implementation choices.

Stage 16: Mobility

Counting options

Two pieces of equal nominal value can be worth very different amounts depending on how many squares they actually move to. A knight on e5 controls eight squares; a knight on a8 controls two. A bishop on an open diagonal sees most of the board; the same bishop blocked behind its own pawn sees almost nothing. Material and PSTs both partially capture this (central PST entries are higher in part because central squares command more), but neither directly counts the moves themselves, and a fully blocked piece on a high-PST square can still be doing nothing useful.

The mobility term is the simplest dynamic feature of a position: how many legal moves does each side have, and what's the difference? A mobile side has options and threats; a cramped side has neither. Empirically, the option count correlates well with positional strength, and the standard implementation in classical evaluators is to count legal moves for both sides and weight the difference at a few centipawns per move. The kit's reference uses `DEFAULT_MOBILITY_WEIGHT = 2`, so a ten-move mobility advantage is worth a fifth of a pawn; this is a deliberately conservative value because at depth one a heavy mobility term encourages active moves the bot cannot defend after the opponent's reply. Larger weights tend to hurt at depth one and help at depth three or beyond, and the right value for your bot depends both on the depth and on how your PSTs and material work alongside it. Phase 5's deeper search lets you crank mobility up; for Phase 4 a small weight or even zero is often best.

There is one wrinkle: the student-facing `Board` exposes `legal_moves()` only for the current side to move. To count the other side's moves too, you have to call `legal_moves` twice with the side temporarily swapped. The pattern is to save `board.state.side_to_move`, set it to the other color, call `legal_moves()`, then restore the saved value. The board's external state ends up unchanged, but in between you get to ask the question the API doesn't directly answer. A `try/finally` around the temporary mutation keeps the restoration reliable even if the move generator raises.

A consequence of this design is that mobility is the most expensive eval term: each call to `legal_moves` traverses every piece on the board generating pseudo-legal moves and then filters them for legality, which is roughly the same cost as one perft step. With material and PSTs your evaluator runs in microseconds; with mobility added it runs in tens or hundreds of microseconds. Phase 4 has only one ply of lookahead, so this is still fast enough to play in real time, but in Phase 5 the cost of a single mobility-aware evaluation will affect how deeply you can search, and many strong engines either omit mobility from the leaf evaluator entirely or compute it from a cheaper proxy (such as counting pseudo-legal moves without the legality filter).

Step 1.

Extend `evaluate` with a mobility term. Compute the number of legal moves for White and the number for Black; the mobility contribution is the difference (White count minus Black count) times your chosen mobility weight, added to the running total. `DEFAULT_MOBILITY_WEIGHT` from `chessdk` is the canonical starting value, but a smaller or larger weight is fine.

To get both sides' move counts you will need to temporarily swap the side. The field to mutate is `board.state.side_to_move`, and the kit relies on it being correctly restored, so wrap the mutation in a `try/finally`. The side-to-move on the original board can use its existing `legal_moves()` call directly; only the other side needs the swap.

Step 2.

Run the Stage 16 tests:

```
chess-cli test -k stage16
```

The tests compare position pairs with identical material and equivalent PST contributions but different mobility: a rook on a central square versus the same rook on the corner, and a position with developed minor pieces versus the same pieces still on the back rank. In each case the more mobile side should evaluate higher.

If a test fails with a near-zero or wrong-sign difference, the usual culprit is calling `legal_moves()` for the side to move and treating that as “the bot’s mobility” regardless of which side the bot is on. The eval is from White’s POV, not from the bot’s POV; positive mobility means *White* has more moves than Black, not that the side to move has more. Make sure the difference is consistently white minus black.

Step 3.

Play the full field of Phase 4 house bots:

```
for opp in materialist nightmare edge_lord crusader hoarder; do
  chess-cli play --vs $opp -n 10
done
```

Crusader is the new aggressive personality. It uses material plus PSTs, plus a large bonus for every one of its own pieces that attacks a square in the 3x3 zone around the enemy king. It does not subtract for being attacked itself; Crusader is single-mindedly offensive, and it will throw material at your king without defending its own. The right counter is mobility plus careful piece coordination; if your bot’s mobility term is healthy, your defensive moves register as positive eval and your bot stays solid through Crusader’s attacks. Bots without a defensive term tend to lose to Crusader because they accept the attacker’s trades.

Hoarder is the slow-game personality. It values material at 1.5× the textbook and adds a small penalty for every legal move available to either side, so it likes positions where there is material on the board and not much to do with it. It refuses trades, retreats pieces to safe squares, and lets the game drift. The counter is active play: your mobility term plus a willingness to push pawns and open files should let your bot pry open the position and drive the game toward a decisive result. A bot without mobility may struggle here, because Hoarder’s slow accumulation is hard to break without piece activity.

Across the field, your bot at the end of Stage 16 should be winning at least as often as it loses against every house bot, with significantly positive margins against Knightmare, Edge Lord, and Hoarder. The Materialist matchup is the tightest because both bots are depth-one material

counters and the games come down to small tactical sequences either side can blunder. Many games still drag out to the move cap as draws; that ceiling is precisely what Phase 5's deeper search lifts.

Summit *Extend your evaluator*

With material, PSTs, mobility, and terminal-position handling, your evaluator already plays recognizable chess. The summit task for this phase is open-ended: pick one or two additional features and add them to `evaluate`. Suggestions, in roughly increasing order of complexity:

Bishop pair. Two bishops are slightly more than the sum of their individual values, because they control complementary diagonals. The standard bonus is 30 to 50 centipawns when a side has both bishops.

Rook on an open file. A rook on a file with no pawns of its own color (semi-open) or no pawns at all (fully open) is significantly more active than the same rook on a closed file. The standard bonus is 10 to 20 centipawns for semi-open and a bit more for fully open.

Passed pawn bonus. A passed pawn is one with no enemy pawns ahead of it on its own file or on the adjacent files; it can promote without being intercepted by an enemy pawn capture, and the closer to promoting it is the more dangerous it gets. The bonus typically increases with rank: maybe 20 cp on rank 4, 40 on rank 5, 80 on rank 6, 160 on rank 7.

Doubled and isolated pawns. A doubled pawn (two pawns of the same color on the same file) creates weaknesses; an isolated pawn (no friendly pawns on adjacent files) has no support. Each is worth a small penalty, roughly 10 to 20 centipawns.

Knight outposts. A knight on the 5th or 6th rank that cannot be attacked by an enemy pawn is enormously strong; it parks itself in the enemy camp and controls eight squares from a position no pawn can challenge.

King safety. A more sophisticated version of what the Crusader does to attack: count the number of enemy attackers on the squares around your king, and apply a penalty proportional to the count. The standard approach is to count attackers in a 3x3 zone around the king and multiply by a safety weight, but the literature contains many more elaborate variants.

Development bonus, opening only. In the first ten or fifteen moves of the game, a bonus for each minor piece that has left its starting square encourages your bot to develop quickly. The bonus tapers to zero by move 15 or so, since the principle ceases to apply once the opening is over.

There is no required feature on this list. Pick what suits the personality you want your bot to develop. Aggressive bots favor king safety and pawn structure; positional bots favor outposts and rook activity; speed-chess bots might add a development bonus to play sharp openings. The arena leaderboard is the final test, and a thoughtfully chosen single feature can outperform a long list of half-tuned ones.

Going further: cuteshchess and the house bots. For day-to-day iteration the right tool is `chess-cli play`, since it loads your `choose_move` directly and runs games as fast as Python can play them. When you want to watch a match between two house bots in a viewer, save a

PGN for later analysis, or run a more rigorous match with proper time controls, `cutechess` (which you installed back in Phase 3, Stage 10) is the standard tool. `Cutechess` spawns each engine as a separate UCI process and talks to them over `stdin/stdout`, exactly the way the arena server does.

The kit's `chess-bot-uci` console script runs whatever `board.py` and `bot.py` it finds in the working directory, which is how `cutechess` drives your own bot. Each house bot also ships as its own standalone executable, so that GUIs (`cutechess-gui`, `Arena`, `Banksia`, and friends) that take only a single command path with no argument field can still point at them. Each house bot installs as `chess-bot-<name>`, with the name written in kebab case: the five Phase 4 bots are reached via `chess-bot-materialist`, `chess-bot-knightmare`, `chess-bot-edge-lord`, `chess-bot-crusader`, `chess-bot-hoarder`, and the three Phase 3 bots via `chess-bot-random-legal`, `chess-bot-always-captures`, `chess-bot-hangs-pieces`. The script form `chess-bot-uci --house <name>` is available too for shell scripting where you would rather not hard-code the name into a binary path.

To run a ten-game match between your bot and Crusader and record the moves as PGN:

```
cutechess-cli \  
-engine name=mybot cmd=chess-bot-uci dir=$PWD proto=uci \  
-engine name=crusader cmd=chess-bot-crusader proto=uci \  
-each tc=10+0.1 \  
-games 10 \  
-pgnout crusader_match.pgn
```

The PGN file opens in any chess viewer (`lichess.org`'s import-game tool, `ChessBase`, the `SCID` family of programs, or any number of mobile apps) so you can step through individual games move by move and watch where Crusader sacrificed material that your bot accepted or declined. To watch two house bots play each other, swap out the `mybot` line for another `chess-bot-<name>` engine; bot personalities show up much more clearly in slow games with proper time controls than in the snap decisions `chess-cli play` extracts.

Checkpoint. By the end of this phase your bot has a real evaluator with material, piece-square tables, mobility, and one or two features you chose yourself, wrapped in an eval-and-pick loop that keeps Phase 3's hang penalty as a depth-one stopgap. It wins comfortably against Knightmare, Edge Lord, and Hoarder, holds its own against Crusader, and is roughly even with Materialist on a long match. On the arena leaderboard it is competing with bots that have similarly rich evaluators, and the ones above you have started to look further ahead than one move. Phase 5 takes on that second axis: we wrap the evaluator in a minimax search that recurses several plies deep and uses alpha-beta pruning to keep the cost manageable. Same evaluator, deeper sight, dramatically stronger play.